Atividade Avaliativa II — N1

Plano de Gerência de Configuração de Software (PGCS)

Projeto: Academia Universal — Landing Page

Autor	Júlio Cesar Maçaneiro
Projeto	Landing page estática em HTML5 e CSS3 (sem JavaScript)
Enfoque	Gerenciamento de Configuração com práticas DevOps
Repositório	academia-universal-landing (GitHub)
Versão do documento	1.0
Data	23/09/2026

Histórico de revisões do documento

Versão	Data	Descrição	Autor
1.0	23/09/2026	Criação do PGCS	Júlio Cesar Maçaneiro

1. Introdução

1.1 Propósito

Este documento define como os arquivos do projeto **Academia Universal — Landing Page** serão identificados, controlados, registrados, verificados e publicados ao longo do seu desenvolvimento. O objetivo é garantir que, a qualquer momento, seja possível saber quais arquivos compõem o projeto, em que versão eles estão e quem, quando e por que alterou cada um.

1.2 Escopo

O plano se aplica a todos os arquivos do repositório do projeto. O produto é um site de página única (*one-page*) totalmente estático, feito **apenas com HTML5 e CSS3**, sem JavaScript, sem framework, sem back-end e sem banco de dados. Por isso, o plano é propositalmente simples e proporcional ao tamanho do projeto.

1.3 Descrição do projeto

A página apresenta a academia e leva o visitante a se matricular. Ela é dividida nas seções: cabeçalho com menu, Hero, Sobre, Planos (Mensal, Trimestral e Anual), Modalidades, Depoimentos, chamada final (CTA), Contato (com formulário validado pelo próprio HTML) e Rodapé. O layout é responsivo e usa variáveis CSS nativas e as fontes Anton e Work Sans, carregadas do Google Fonts.

1.4 Definições

Termo	Significado
GCS / SCM	Gerência (ou Gerenciamento) de Configuração de Software: disciplina que controla as mudanças nos itens do projeto.
PGCS	Plano de Gerência de Configuração de Software: este documento.
IC	Item de Configuração: qualquer arquivo ou documento que será versionado e controlado.
Baseline	Versão aprovada e marcada de um conjunto de ICs, usada como ponto de referência.
Commit	Registro de uma mudança no Git, com autor, data e mensagem.
Branch	Linha independente de desenvolvimento dentro do repositório.
Tag	Rótulo fixo que marca uma versão (ex.: v1.1.1) no histórico do Git.
Deploy	Publicação da versão do site em um ambiente acessível ao público.
CI/CD	Integração e entrega contínuas: automação do caminho entre o código e a publicação (prática DevOps).

2. Gerenciamento da GCS

2.1 Papéis e responsabilidades

Como a atividade é individual, o mesmo integrante acumula todos os papéis abaixo.

Papel	Responsabilidades	Responsável
Desenvolvedor	Escrever e alterar o HTML e o CSS; testar as mudanças localmente.	Júlio Cesar Maçaneiro
Gerente de Configuração	Definir e manter este plano, os nomes, as versões, as baselines e o registro de mudanças.	Júlio Cesar Maçaneiro
Responsável por release e deploy	Criar as tags de versão e garantir a publicação no GitHub Pages.	Júlio Cesar Maçaneiro
Auditor	Conferir se o repositório e o site estão de acordo com este plano.	Júlio Cesar Maçaneiro

2.2 Ferramentas e ambiente

Ferramenta	Uso no projeto
Git	Controle de versão local (commits, branches e tags).
GitHub	Repositório remoto, histórico público, Issues e Pull Requests.
GitHub Pages	Hospedagem e deploy do site estático, a partir da branch `main`.
Visual Studio Code	Editor de código.
Navegador (Chrome/Firefox) e DevTools	Testes visuais, de responsividade e do formulário.
Servidor local (opcional)	`python3 -m http.server 8000` ou `npx http-server .`
Validadores W3C	Validação do HTML e do CSS.
Google Fonts (dependência externa)	Fontes Anton e Work Sans. Não é versionada no repositório; o CSS define fontes alternativas caso ela falhe.

3. Identificação da configuração

3.1 Itens de configuração

ID	Item	Caminho	Descrição
IC01	Página principal	index.html	Estrutura e conteúdo de todas as seções da landing page.
IC02	Folha de estilos	css/style.css	Todo o visual e a responsividade: cores, tipografia, layout e menu mobile.
IC03	Documentação do projeto	README.md	Descrição, tecnologias, estrutura e como executar.
IC04	Plano de GCS	docs/PGCS.pdf	Este documento.

3.2 Estrutura do repositório

```
academia-universal-landing/
|-- index.html
|-- css/
|   `-- style.css
|-- docs/
|   `-- PGCS.pdf
`-- README.md
```

3.3 Convenções de nomes

- **Arquivos e pastas:** letras minúsculas, sem espaços e sem acentos (ex.: `style.css`). A página inicial se chama sempre `index.html`.
- **Classes CSS:** minúsculas, em português, separadas por hífen (ex.: `plano-destaque`, `hero-actions`).
- **Branches:** `main` (versão estável e publicada), `feature/descricao` (novidades) e `fix/descricao` (correções).
- **Tags:** `vMAJOR.MINOR.PATCH` (ex.: `v1.1.1`).
- **Mensagens de commit:** `tipo: descrição curta no imperativo`, com os tipos `feat` (novidade), `fix` (correção), `style` (visual), `docs` (documentação) e `chore` (manutenção). Ex.: `fix: ajustar horário de funcionamento para 23h`.

3.4 Versionamento

Será usado o versionamento semântico (**MAJOR.MINOR.PATCH**):

Parte	Quando aumenta	Exemplo
MAJOR	Mudança grande de estrutura ou de tecnologia.	1.0.0 para 2.0.0
MINOR	Nova seção ou mudança relevante de comportamento.	1.0.0 para 1.1.0
PATCH	Correção de texto, dado ou detalhe visual.	1.1.0 para 1.1.1

3.5 Baselines

Cada versão publicada recebe uma tag no Git e passa a ser uma baseline. As baselines do projeto estão listadas na seção 4.3.

4. Controle de configuração

4.1 Processo de mudança

Toda alteração em um item de configuração segue os passos abaixo:

Passo	Atividade
1	**Solicitar:** identificar a necessidade (correção, melhoria ou ajuste de conteúdo) e registrá-la como Issue no GitHub.
2	**Isolar:** criar uma branch a partir da `main` (`feature/...` ou `fix/...`).
3	**Implementar e testar:** alterar os arquivos e testar no navegador (computador e celular), conferindo links do menu, layout e formulário.
4	**Registrar:** fazer o commit seguindo a convenção de mensagens.
5	**Revisar:** abrir um Pull Request e revisar o que mudou (*diff*) antes de aprovar.
6	**Integrar:** fazer o merge na `main` e apagar a branch.
7	**Versionar:** criar a nova tag, atualizar o registro de mudanças (4.3) e o README, se necessário.
8	**Publicar:** o GitHub Pages atualiza o site automaticamente a partir da `main`.

4.2 Política de branches

- A `main` deve estar sempre funcional, pois é ela que é publicada.
- Mudanças são feitas em branches separadas e só entram na `main` depois de testadas.
- Nunca reescrever o histórico da `main`. Para desfazer algo, usar `git revert`.

4.3 Registro de mudanças e baselines

ID	Data	Mudança	Itens afetados	Versão (tag)
MUD-01	22/09/2026	Criação inicial da landing page (com JavaScript para menu mobile, validação do formulário e botão voltar ao topo).	IC01, IC02, IC03 e js/script.js	v1.0.0
MUD-02	23/09/2026	Remoção do JavaScript: menu passa a ser uma linha de links no celular, validação feita pelo HTML e ano do rodapé fixo. Removidos o botão voltar ao topo e o arquivo js/script.js.	IC01, IC02, IC03	v1.1.0
MUD-03	23/09/2026	Horário de funcionamento alterado de 05h–24h para 05h–23h.	IC01	v1.1.1
MUD-04	23/09/2026	Inclusão do PGCS no repositório.	IC04	v1.1.1

5. Relatório de status da configuração

O status dos itens é consultado no próprio Git e no GitHub, sem ferramenta adicional:

Necessidade	Como consultar
Estado atual dos arquivos	`git status`
Histórico de mudanças	`git log --oneline`, ou a aba Commits no GitHub
Versões publicadas	`git tag`, ou a aba Releases no GitHub
O que mudou entre versões	`git diff v1.1.0 v1.1.1`
Quem alterou cada linha	`git blame index.html`

Situação dos itens de configuração na versão atual:

ID	Item	Versão atual	Status
IC01	index.html	v1.1.1	Aprovado e publicado
IC02	css/style.css	v1.1.1	Aprovado e publicado
IC03	README.md	v1.1.1	Aprovado
IC04	docs/PGCS.pdf	v1.1.1	Aprovado

6. Auditorias e revisões

Antes de cada tag de versão é feita uma auditoria em dois níveis:

- **Auditoria funcional:** verifica se o site funciona como o esperado.
- **Auditoria física:** verifica se o repositório está de acordo com este plano (estrutura, nomes, tags, README e registro de mudanças).

Verificação	Tipo	Resultado
O `index.html` abre sem erros no navegador	Funcional	
Todos os links do menu e dos botões levam à seção correta	Funcional	
O layout se adapta a computador, tablet e celular	Funcional	
O formulário exige nome, e-mail válido e telefone com DDD	Funcional	
O HTML e o CSS passam nos validadores do W3C	Funcional	
O site publicado no GitHub Pages abre e está igual à `main`	Funcional	
Não existe nenhum arquivo `.js` no repositório	Física	
A estrutura de pastas e os nomes seguem a seção 3	Física	
Commits e tags seguem as convenções; a versão atual tem tag	Física	
README e registro de mudanças estão atualizados	Física	

O campo Resultado é preenchido (Conforme / Não conforme) a cada auditoria. Itens não conformes geram uma nova solicitação de mudança.

7. Build, release e deploy (DevOps)

Por ser um site estático, não há etapa de compilação: os próprios arquivos do repositório são o produto final. O fluxo de entrega é:

Etapa	Descrição
Commit	A mudança é registrada em uma branch.
Pull Request e merge	A mudança é revisada e integrada à `main`.
Release	É criada a tag da versão (ex.: `git tag v1.1.1` e `git push --tags`).
Deploy	O GitHub Pages (Settings > Pages > branch `main`, pasta raiz) publica a nova versão automaticamente a cada push.
Rollback	Em caso de problema, usar `git revert` ou publicar novamente a tag anterior.

Nesta versão do projeto a publicação é automática (entrega contínua), mas os testes são manuais, conforme a seção 6. Como evolução futura, pode-se adicionar um fluxo do GitHub Actions que valide o HTML e o CSS a cada push.

8. Backup e recuperação

O repositório remoto no GitHub funciona como backup, e cada computador que clona o projeto guarda uma cópia completa do histórico. Para recuperar o projeto ou uma versão antiga:

```
git clone <url-do-repositorio>
git checkout v1.1.0
```

9. Riscos de configuração

Risco	Mitigação
Perda dos arquivos locais	Push frequente para o GitHub.
Alteração com erro na `main` deixar o site quebrado	Trabalhar em branches, testar antes do merge e usar `git revert`.
Versões diferentes sem controle	Tags de versão e registro de mudanças atualizado.
Google Fonts fora do ar	O CSS define fontes alternativas (Arial Narrow e fontes do sistema).
Documentação diferente do repositório	Auditoria física antes de cada release.

10. Referências

- IEEE Std 828: Standard for Configuration Management in Systems and Software Engineering.
- CHACON, S.; STRAUB, B. *Pro Git*. 2. ed. Apress, 2014. Disponível em git-scm.com/book.
- Semantic Versioning 2.0.0. Disponível em semver.org.
- GitHub Docs: GitHub Pages. Disponível em docs.github.com/pages.
- W3C Markup Validation Service e W3C CSS Validation Service.